

Requirement Analysis Document

1. Vision

The aim is to develop a modular Retrieval-Augmented Generation (RAG) chatbot system that answers students' questions about the Department of Computer Engineering at Marmara University and the faculty regulations, working entirely on local documents. The system will operate in a deterministic and traceable manner without requiring any external APIs.

2. Goals

- To construct a local knowledge base generated from text-based documents that have been normalized and chunked.
- To establish an architecture that adheres to GRASP and SOLID principles.
- To operate without a GUI or database, using only the CLI and YAML/JSON files for configuration, storage, and execution workflows.
- To make the query processing and answer generation pipelines interchangeable through the Strategy design pattern.

3. Glossary

- **RAG:** Retrieval-Augmented Generation; the process of retrieving information and generating answers.
- **Chunk:** Small text blocks created by splitting documents.
- **Intent:** The underlying purpose behind the user's question.
- **Stopwords:** Common words ignored during search.
- **CLI:** A text-based interface for executing commands.
- **Queries:** Text-based search requests sent to retrieve information.
- **Heuristic:** A simple rule that gives quick but not necessarily perfect results.

4. Risks

- Keyword matching being insufficient for complex user queries
- If documents are incorrectly segmented, the chunking stage may break and produce incorrect answers
- Very large or very small chunks causing loss of semantic
- JSON/YAML slowing down when handling large text data

5. NFR(Non-Functional Requirement)

- **Testability:** Each component (IntentDetector, Retriever, etc.) must be unit-testable in isolation.
- **Traceability:** Every step of the system (input/output/duration) must be loggable in JSONL format.
- **Constraints:** The system must not use a GUI or a database; it should operate only with a CLI and JSON/YAML files.
- **Performance:** The system must not rely on internet access or external API calls.
- **Maintainability:** Code must allow feature updates within 1 day

6. Use Cases

6.1. Search for Staff Information

Use Case Name: Search for Staff Information

Scope: MiniRAG Chatbot System (CLI Application)

Level: User-goal

Actors:

- Primary: Student
- Secondary: None

Stakeholders and Interests:

- Student: Wants to quickly and accurately obtain the instructor's office/email information.
- Instructor: Does not want to receive frequent emails for simple questions such as office hours or location, prefers students to access the information via the system.

Preconditions:

- The system (JAR file) must be runnable.
- config.yaml must contain valid paths.
- Text files containing staff data must be indexed in the KeywordIndex.

Postconditions:

- The system outputs the answer and the relevant document citation to the terminal (stdout).

Main Success Scenario:

1. The student asks: "Ali Haydar Özer'in maili nedir?"
2. IntentDetector: Scans keywords in the question such as "instructor," "email," and "what" and determines the intent as **StaffLookup**
3. QueryWriter: Normalizes the question; since the intent is StaffLookup, adds booster terms to the query.
4. Retriever: Finds the most relevant chunks from the index based on term frequency.
5. Reranker: If the chunk title contains the instructor's name, applies a TitleBoost to update the ranking.
6. AnswerAgent: Selects the most relevant sentence from the highest-scoring chunk.
7. The system formats the response: "Your answer: [Sentence]. See: [DocID:Section]"
8. The entire process is logged via TraceBus.

Extensions:

- 2a. If Intent Cannot Be Determined:
 - The system marks the intent as Unknown.
 - QueryWriter only performs stopwords removal and does not add special terms.
 - The process continues, but answer quality may decrease.
- 4a. If No Document Can Be Found:
 - Retriever returns an empty list.

- AnswerAgent outputs: "Sorry, I could not find information on this topic."

Special Requirements:

- Determinism: The system must always produce the same answer for the same question and configuration.

Technology and Data Variations List:

- Input is provided only via CLI arguments.
- Data source is limited to local .txt or .json files.
- Frequency of Occurrence: High (especially at the start of the term and during exam periods).
- Miscellaneous: In Iteration 1, since no LLM is used, answers are not “human-like” but are direct text excerpts.

6.2. Search for Regulation

Use Case Name: Query University Regulation

Scope: MiniRAG Chatbot System (CLI Application)

Level: User-goal

Actors:

- Primary: Student

Stakeholders and Interests:

- Student: Wants to understand their rights or rules without getting lost in regulations.
- University Administration: Wants rules to be conveyed to students correctly and in a standardized manner.

Preconditions:

- M.U. regulations must be downloaded, chunked, and indexed.
- stopwords.yaml must include irrelevant words.

Postconditions:

- The system outputs the sentence containing the relevant regulation clause along with the document ID to the terminal.

Main Success Scenario:

1. Student asks: “How many courses are students allowed a single-course exam for?”
2. IntentDetector: Detects intent as PolicyFAQ (or Registration) from keywords “exam,” “right,” and “course.”
3. QueryWriter: Removes question root words and stems or keeps terms like “single course,” “exam,” “right.”
4. Retriever: Finds the chunk with the highest frequency of these terms.
5. Reranker: If terms are close together, moves the paragraph to the top of the list.
6. AnswerAgent: Extracts the sentence from the best paragraph that contains the query terms.

7. System responds: "Your answer: For graduation, students with up to three remaining courses... See: mazeret_yonergesi.txt:Madde5"
8. TraceBus: Logs the event.

Extensions:

- 5a. Tie Results:
 - Reranker applies a deterministic rule: first alphabetically by DocID, then by ChunkID.
- 6a. Sentence Cannot Be Selected:
 - The first sentence of the best chunk is returned by default.

Special Requirements:

- Traceability: The source regulation file must be clearly indicated.

Technology and Data Variations List:

- Data is read from the folder specified in config.yaml.
- Frequency of Occurrence: Medium (usually increases during final exam periods).
- Miscellaneous: Since keyword matching is used, students must use the exact terms from the regulation (e.g., "single course"); synonyms cannot be recognized in Iteration 1.